Java Streams & Lambda Expressions

1. What is a Lambda Expression?

- A lambda expression is a short way to represent an anonymous function (a function without a name).
- Introduced in Java 8, it allows us to pass behavior as a parameter.
- It is mainly used with functional interfaces.

2. Why use Lambdas?

- Makes code shorter and more readable.
- Helps in writing functional style code.
- Useful for passing behavior as an argument.
- Widely used with Streams and collection operations.

3. Basic Syntax

- (parameters) → expression
- (parameters) → { statements }

4. Common Forms

Form	Example
No parameter	() → System.out.println("Hello");
Single parameter	x → x * 2
Multiple parameters	(a, b) → a + b
Block lambda	(x) → { System.out.println(x); }
Return lambda	(x) → x * x

5. Simple Examples

1. Runnable (no parameter) : Runnable r = () → System.out.println("Running");
2. Square a number : Function<Integer, Integer> square = x → x * x;
3. Add two numbers : BiFunction<Integer, Integer, Integer> add = (a, b) → a + b;
4. Print a name : Consumer<String> printName = name → System.out.println(name);

6. Lambda vs Normal Method

Feature	Lambda Expression	Normal Method
Name	No name (anonymous)	Has a name
Syntax	Short and concise	More verbose
Used for	Passing behavior as a value	Defining reusable functionality
Suitable for	Functional interfaces	General purpose use
Example	x → x * x	int square(int x) { return x * x; }

7. Where Lambdas are used?

- Collections (e.g., forEach, removeIf, sort)
- Streams (e.g., filter, map, reduce)
- Threads (e.g., Runnable)
- Event handling (e.g., button click listeners in GUI)
- Functional interfaces (e.g., Predicate, Function, Consumer, Supplier)

Tip : A lambda expression can be used wherever a functional interface is expected.

Java Streams & Lambda Expressions

Functional Interfaces & Built-in Interfaces

1. What is a Functional Interface?

- An interface that contains exactly one abstract method.
- It can have default and static methods (these do not count).
- Used as the target type for lambda expressions and method references.

2. @FunctionalInterface

- An annotation to indicate that an interface is intended to be functional.
- Helps the compiler give an error if more than one abstract method exists.
- It is optional, but recommended for clarity.

3. Common Built-in Interfaces (in java.util.function)

- Predicate<T> : Takes T, returns boolean
- Function<T, R> : Takes T, returns R
- Consumer<T> : Takes T, returns nothing (void)
- Supplier<T> : Takes nothing, returns T
- UnaryOperator<T> : Takes T, returns T (special case of Function)
- BinaryOperator<T> : Takes two T, returns T (special case of BiFunction)
- Comparator<T> : Compares two T, returns int

4. Quick Reference Table

Interface	Method	Use
Predicate<T>	boolean test(T t)	Filtering / condition check
Function<T, R>	R apply(T t)	Transform input to output
Consumer<T>	void accept(T t)	Perform an action (no return)
Supplier<T>	T get()	Provide a value (no input)
UnaryOperator<T>	T apply(T t)	Transform and return same type
BinaryOperator<T>	T apply(T t1, T t2)	Combine two values
Comparator<T>	int compare(T a, T b)	Compare two values (sorting)

5. Examples

```
Predicate<Integer> isEven = n -> n % 2 == 0;          // true for even numbers
Function<String, Integer> len = s -> s.length();      // length of a string
Consumer<String> print = s -> System.out.println(s);  // print a value
Supplier<Double> random = () -> Math.random();        // supply a random number
```

6. Custom Functional Interface

```
@FunctionalInterface
interface Greeting {
    void sayHello(String name);
}

// Using lambda
Greeting g = name -> System.out.println("Hello, " + name);
g.sayHello("Java"); // Output: Hello, Java
```

7. Key points / Remember

- A functional interface has exactly one abstract method.
- Lambda expressions provide a compact way to implement it.
- Many useful functional interfaces are available in java.util.function.
- They are widely used in Streams, event handling, and functional-style programming.

Mini tip :

Lambda expression = implementation of a functional interface.

Java Streams & Lambda Expressions

Method References

1. What is a Method Reference?

A method reference is a shorthand notation for a lambda expression that just calls an existing method. It makes the code more readable and concise.

2. Syntax

ClassName::methodName	:	Reference to a static method
objectName::methodName	:	Reference to an instance method of a particular object
ClassName::methodName	:	Reference to an instance method of an arbitrary object
ClassName::new	:	Reference to a constructor

3. Types of Method References

1. Static method reference → ClassName::methodName
2. Instance method reference (particular object) → objectName::methodName
3. Instance method reference (arbitrary object) → ClassName::methodName
4. Constructor reference → ClassName::new

4. Small examples

```
1. Integer::parseInt        // reference to static method
2. System.out::println      // reference to instance method of a particular object
3. String::toLowerCase      // reference to instance method of an arbitrary object
4. ArrayList::new           // reference to constructor
```

5. Lambda vs Method Reference

Example	Lambda Expression	Method Reference
1. Parse Integer	s → Integer.parseInt(s)	Integer::parseInt
2. Print to console	x → System.out.println(x)	System.out::println
3. To lower case	s → s.toLowerCase()	String::toLowerCase
4. Create list	() → new ArrayList<>()	ArrayList::new

6. Where useful?

1. forEach() – e.g. list.forEach(System.out::println)
2. map() – e.g. list.stream().map(String::toLowerCase)
3. sorting – e.g. list.stream().sorted(String::compareTo)
4. object creation – e.g. list.stream().map(ArrayList::new)

7. Quick tips / Remember

1. Use method references when a lambda only calls an existing method.
2. They make code shorter and more readable.
3. The method signature must match the functional interface.
4. Constructors can also be referenced using ClassName::new.

Java Streams & Lambda Expressions

1. What is a Stream?

- A Stream is a sequence of elements that supports functional-style operations on those elements.
- Key idea: Process data, not store data.

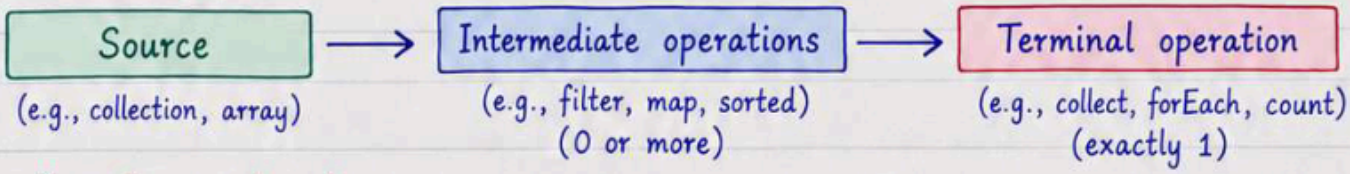
2. Why use Streams?

- Clean and readable data processing.
- Less boilerplate code (fewer loops).
- Functional style (use lambda expressions).
- Supports operations like filter, map, reduce, etc.

3. Stream vs Collection

Feature	Collection	Stream
Purpose	Stores data	Processes data
Data storage	Stores elements in memory	Does not store elements
Operations	Add, remove, iterate, etc.	Filter, map, reduce, etc.
When to use	When you need to store data	When you need to process data
Reusable	Yes (can be used multiple times)	No (single use)

4. Stream pipeline



(e.g., collection, array)

(e.g., filter, map, sorted)
(0 or more)

(e.g., collect, forEach, count)
(exactly 1)

5. How to create streams

1. From a Collection: `list.stream()`
2. From an array: `Arrays.stream(array)`
3. Using Stream.of(): `Stream.of(element1, element2, ...)`
4. From files: `Files.lines(Paths.get("file.txt"))`
5. From numbers: `IntStream.range(1, 10)`

6. Short examples

1.
```
List<String> list = Arrays.asList("a", "b", "c");
list.stream();
```
2.
```
String[] arr = {"a", "b", "c"};
Arrays.stream(arr);
```
3.
```
Stream.of("a", "b");
```

7. Important properties

1. Does not store elements.
2. Lazy – operations are not executed until a terminal operation is invoked.
3. Uses internal iteration (handled by the framework).
4. Not reusable – a stream can be used only once (single use).

8. Remember:

A stream is for processing data, not for storing data.
Build a pipeline, apply a terminal operation, and you're done!

Java Streams & Lambda Expressions

Intermediate Operations – Part 1

1. What are intermediate operations?

- Intermediate operations are lazy operations that return a stream.

2. filter()

- Filters elements based on a condition.
- Example:

```
List<Integer> even = numbers.stream()
                            .filter(n -> n % 2 == 0)        // [2, 4, 6]
                            .collect(Collectors.toList());
```

3. map()

- Transforms each element into another form.
- Example:

```
List<String> names = List.of("java", "stream", "lambda");
List<String> upper = names.stream()
                          .map(String::toUpperCase)
                          .collect(Collectors.toList());  // [JAVA, STREAM, LAMBDA]
```

4. flatMap()

- Maps each element to a stream and flattens the result.
- Example:

```
List<List<Integer>> list = List.of(List.of(1, 2), List.of(3, 4));
List<Integer> flat = list.stream()
                         .flatMap(Collection::stream)
                         .collect(Collectors.toList());   // [1, 2, 3, 4]
```

5. distinct()

- Removes duplicate elements.
- Example:

```
List<Integer> nums = List.of(1, 2, 2, 3, 3, 4);
List<Integer> unique = nums.stream()
                           .distinct()
                           .collect(Collectors.toList());  // [1, 2, 3, 4]
```

6. sorted()

- Returns elements in sorted order.
- Examples:

```
List<Integer> nums = List.of(5, 1, 4, 2);
List<Integer> sorted = nums.stream().sorted().collect(Collectors.toList()); // [1, 2, 4, 5]
List<String> words = List.of("banana", "apple", "cherry");
List<String> custom = words.stream()
                           .sorted(Comparator.comparingInt(String::length)) // [apple, banana, cherry]
                           .collect(Collectors.toList());
```

7. Summary Table

Method	Purpose	Sample Output
filter()	Selects elements that match a condition	[2, 4, 6]
map()	Transforms each element	[JAVA, STREAM, LAMBDA]
flatMap()	Flattens streams of streams	[1, 2, 3, 4]
distinct()	Removes duplicates	[1, 2, 3, 4]
sorted()	Sorts elements (natural or custom)	[1, 2, 4, 5] / [apple, banana, cherry]

Mini tip : Intermediate operations are lazy and are not executed until a terminal operation is called (e.g., collect(), forEach(), count()).

Java Streams & Lambda Expressions

Intermediate Operations – Part 2

1. limit()

- Returns only first n elements from the stream.
- Example :

```
List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5);
nums.stream().limit(3).forEach(System.out::println);
// Output: 1 2 3
```

2. skip()

- Skips first n elements from the stream.
- Example :

```
List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5);
nums.stream().skip(2).forEach(System.out::println);
// Output: 3 4 5
```

3. peek()

- Performs an action on each element and returns the same stream.
- Useful for debugging (not for modifying data).
- Example :

```
nums.stream()
    .peek(n -> System.out.println("Value: " + n))
    .forEach(System.out::println);
```

4. mapToInt / mapToDouble / mapToLong

- Convert a stream of objects to a primitive stream (IntStream, DoubleStream, LongStream).
- Avoids boxing and improves performance for numeric operations.
- Example :

```
List<String> nums = Arrays.asList("1", "2", "3");
int sum = nums.stream().mapToInt(Integer::parseInt).sum();
```

5. boxed()

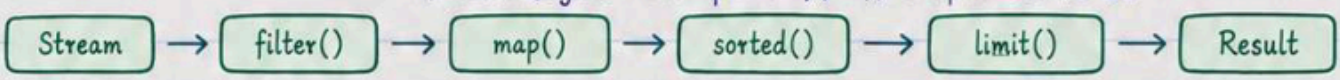
- Converts a primitive stream to a stream of wrapper objects.
- Example :

```
IntStream intStream = IntStream.of(1, 2, 3);
Stream<Integer> boxed = intStream.boxed();
```

6. Chaining example

- Combine multiple intermediate operations.
- Example :

```
List<Integer> nums = Arrays.asList(5, 1, 8, 3, 9, 2);
nums.stream()
    .filter(n -> n % 2 == 0)   // keep even numbers
    .map(n -> n * 10)          // multiply by 10
    .sorted()                  // sort ascending
    .limit(3)                  // take first 3
    .forEach(System.out::println);  // Output: 20 40 80
```



7. Common mistakes

- Modifying source in peek() → Don't change the original collection inside peek().
- Assuming execution without terminal operation → Intermediate operations are lazy.

8. Key note / Remember

Remember: Intermediate operations return a new stream and are lazy. They only execute when a terminal operation is called.

Java Streams & Lambda Expressions

Terminal Operations

1. What are terminal operations?

Terminal operations close / consume the stream and produce a result (or a side-effect). After a terminal operation, the stream cannot be reused.

2. forEach()

- Use : Performs an action for each element (usually for side-effects).
- Example :

```
List<String> names = List.of("Aman", "Pooja", "Rohit");
names.stream().forEach(name -> System.out.println(name));
```

3. collect()

- Use : Collects stream elements into a collection (e.g., List, Set, Map).
- Example :

```
List<String> names = List.of("Aman", "Pooja", "Rohit");
List<String> list = names.stream().collect(Collectors.toList());
```

4. reduce()

- Use : Combines elements into a single result.
- Example :

```
List<Integer> numbers = List.of(1, 2, 3, 4, 5);
int sum = numbers.stream().reduce(0, (a, b) -> a + b); // sum = 15
```

5. count(), min(), max(), findFirst(), findAny()

- Example :

```
List<Integer> numbers = List.of(5, 2, 9, 1, 7);
long count              = numbers.stream().count();                    // 5
Optional<Integer> min   = numbers.stream().min(Integer::compareTo); // 1
Optional<Integer> max   = numbers.stream().max(Integer::compareTo); // 9
Optional<Integer> first = numbers.stream().findFirst();                // 5
Optional<Integer> any   = numbers.stream().findAny();                  // e.g. 5
```

6. anyMatch(), allMatch(), noneMatch()

- Example :

```
List<Integer> numbers = List.of(2, 4, 6, 7);
boolean anyEven  = numbers.stream().anyMatch(n -> n % 2 == 0);  // true
boolean allEven  = numbers.stream().allMatch(n -> n % 2 == 0);  // false
boolean noneEven = numbers.stream().noneMatch(n -> n % 2 == 0); // false
```

7. toArray()

- Example :

```
List<String> names = List.of("Aman", "Pooja", "Rohit");
String[] arr = names.stream().toArray(String[]::new);
```

8. Summary Table

Method	Return type	Use
forEach()	void	Performs an action on each element
collect()	Collection (e.g., List)	Collects elements into a collection
reduce()	T (single value)	Combines elements into a single result
count()	long	Counts number of elements
min() / max()	Optional<T>	Returns minimum / maximum element
findFirst() / findAny()	Optional<T>	Returns first element / any element
anyMatch()	boolean	True if any element matches the condition
allMatch()	boolean	True if all elements match the condition
noneMatch()	boolean	True if no elements match the condition
toArray()	Array (e.g., T[])	Converts elements to an array

Remember: After a terminal operation, the stream cannot be reused.

8. Collectors

1. What is Collectors?

Collectors is a utility class in java.util.stream that provides factory methods for mutable reduction (collecting stream elements).

2. Common collectors

- toList() : Collects elements into a List
- toSet() : Collects elements into a Set (no duplicates)
- toMap() : Collects elements into a Map (key → value)

3. joining() - example

```
String result = List.of("Java", "is", "fun")
                    .stream().collect(Collectors.joining(" "));
// result = "Java is fun"
```

4. counting(), summingInt(), averagingInt()

```
List<Integer> nums = List.of(1, 2, 3, 4, 5);
long count = nums.stream().collect(Collectors.counting()); // 5
int sum = nums.stream().collect(Collectors.summingInt(n → n)); // 15
double avg = nums.stream().collect(Collectors.averagingInt(n → n)); // 3.0
```

5. groupingBy() - example

```
List<Employee> employees = ...;
Map<String, List<Employee>> byDept = employees.stream()
                    .collect(Collectors.groupingBy(Employee::getDepartment));
```

6. partitioningBy() - example

```
List<Integer> nums = List.of(1, 2, 3, 4, 5, 6);
Map<Boolean, List<Integer>> evenOdd = nums.stream()
                                .collect(Collectors.partitioningBy(n → n % 2 == 0));
// true → [2, 4, 6], false → [1, 3, 5]
```

7. mapping() / collectingAndThen() - example

```
List<String> names = employees.stream()
                    .collect(Collectors.mapping(Employee::getName, Collectors.toList()));
List<String> unmodifiable = names.stream()
                .collect(Collectors.collectingAndThen(Collectors.toList(), List::of)); // returns unmodifiable list
```

8. Common collectors table

Collector	Output	Use case
toList()	List<T>	Collect elements into a List
toSet()	Set<T>	Collect elements into a Set (no duplicates)
toMap(k, v)	Map<K, V>	Collect elements into a Map
joining(delim)	String	Join string elements
counting()	Long	Count number of elements
summingInt()	Integer	Sum of integer values
averagingInt()	Double	Average of integer values
groupingBy()	Map<K, List<T>>	Group elements by a key
partitioningBy()	Map<Boolean, List<T>>	Partition elements into two groups (true/false)
mapping()	Transformed type	Apply a mapping function before collecting
collectingAndThen()	R (final type)	Post-process the result of a collector

Remember:

- Collectors are designed for use with Stream.collect(...).
- Use the right collector based on the result type you need.
- groupingBy and partitioningBy help organize data, while mapping and collectingAndThen help transform results.

Java Streams & Lambda Expressions

Optional & Real Project Usage

1. What is Optional?

- Optional is a container object that may or may not contain a value.
- It helps to avoid null checks and reduce NullPointerException.

2. Creating Optional

1. Optional.of(value)
   → Creates Optional with a non-null value
2. Optional.ofNullable(value)
   → Creates Optional, can be null
3. Optional.empty()
   → Creates an empty Optional

3. Useful methods

1.	isPresent()	: Returns true if value exists
2.	ifPresent()	: Executes code if value exists
3.	orElse()	: Returns value or default value
4.	orElseGet()	: Returns value or value from supplier
5.	orElseThrow()	: Returns value or throws exception
6.	map()	: Transforms the value if present

4. Short Optional examples

```
1. Optional<String> name =
   Optional.of("Aman");
2. Optional<String> empty = Optional.empty();
3. Optional<String> value = Optional.ofNullable(null);
4. value.isPresent();              // false
5. value.orElse("Unknown");        // "Unknown"
6. value.ifPresent(v ->
       System.out.println(v));     // no output
```

5. Real project usage of Streams + Lambdas

1.	Filtering active users	: users.stream().filter(User::isActive)
2.	Sorting invoices	: invoices.stream().sorted(...)
3.	Mapping DTOs	: users.stream().map(User::toDto)
4.	Grouping orders	: orders.stream().collect(groupingBy(...))
5.	Finding duplicates	: items.stream().collect(groupingBy(...))

6. Real project style example

```
// Get active user emails, sorted and mapped to lowercase
List<String> emails = users.stream()
        .filter(User::isActive)
        .map(User::getEmail)
        .filter(Objects::nonNull)
        .map(String::toLowerCase)
        .sorted()
        .collect(Collectors.toList());
```

7. Common mistakes

1. Overusing Optional in fields/parameters
   - Avoid using Optional as a class field or method parameter.
   - Use it mainly for return types.
2. Writing complex stream chains
   - Too many operations can reduce readability.
   - Break into multiple steps if needed.

8. Key tips / remember

1. Use Optional to avoid null checks, not everywhere.
2. Prefer method references (User::getName).
3. Keep stream operations simple and readable.
4. Use meaningful variable names.
5. Use collectors like toList(), toSet(), groupingBy().
6. Streams are powerful, but readability matters.
7. Test edge cases (empty lists, null values).

Java Streams & Lambda Expressions

Parallel Streams, Best Practices & Interview Quick Revision

1. What is parallelStream()?

parallelStream() creates a parallel stream that processes elements in multiple threads (using the ForkJoinPool) to speed up operations.

2. stream() vs parallelStream()

Aspect	stream()	parallelStream()
Order	Maintains encounter order	May not maintain order
Performance	Sequential processing	Can be faster for large data
Thread usage	Single thread	Multiple threads (ForkJoinPool)
Use case	Small or simple collections	Large, CPU intensive tasks

3. When to use parallel streams

1. CPU heavy operations (e.g., complex calculations, data processing)
2. Large collections
3. Independent operations (no dependency between elements)

4. When not to use

1. I/O bound tasks (e.g., reading files, network calls)
2. Shared mutable state (can cause race conditions)
3. Small collections (overhead may be higher than benefit)

5. Best practices

1. Keep lambdas small and focused
2. Prefer method references when readable
3. Avoid side effects in lambdas
4. Avoid reusing streams (they can be used only once)
5. Use clear and appropriate collectors (e.g., toList(), toSet(), groupingBy())

6. Interview questions quick revision

1. What is a stream? → A sequence of elements supporting functional operations.
2. What is lazy evaluation? → Intermediate operations are not executed until a terminal operation is called.
3. Difference between filter() and map()? → filter() selects elements, map() transforms them.
4. What does reduce() do? → Combines elements into a single result.
5. What is a functional interface? → An interface with exactly one abstract method (e.g., Runnable, Function).
6. What is Optional? → A container object to handle possible null values.

7. Final remember (Key takeaways)

1. parallelStream() can speed up processing using multiple threads.
2. Use it for large, CPU heavy, independent tasks.
3. Avoid it for I/O, shared state, or small collections.
4. Write clean, small, side-effect-free lambdas and use method references.
5. Streams are lazy, fluent and powerful — understand the core operations well.